Bucharest Apartment Rent Prediction

Technical Report Analysis on Regression Models

Justin Marian Popescu

May 1, 2026

Abstract

End-to-end regression pipeline for estimating the rent per month (in EUR) of apartments in Bucharest is presented. It involves structured tabular features, like area, number of rooms, floor, build year, building features, and unstructured text features, such as tags, amenities, and location information. The whole pipeline is described from exploratory data analysis (EDA), preprocessing, feature engineering, and training and evaluating regression algorithms, namely, a Ridge linear regression baseline, a single decision tree, Random Forest regression, and gradient-boosted trees using XGBoost. After that, an ablation study is conducted to assess the importance of log-target transformation, engineered features, and encoding strategy for categorical features, and finally, hyperparameter tuning is performed on the best-performing models. The target metric is the root mean square error (RMSE) in EUR. The best model is a tuned XGBoost regressor, yielding ≈ 301 EUR CV RMSE; the tuned+deep boosting blend follows at the same level, with a safety margin on a 20% holdout split.

1 Problem Setting and Data

The objective is the linear regression of `price` in monthly rent (in EUR) for a collection of $\sim 9\,500$ Bucharest apartment ads. The data is provided in this format including `train.csv`, `test.csv`, and `sample_submission.csv` files; this data includes ~ 50 heterogenous features comprising numerical variables (`area_m2`, `rooms`, `floor`, `build_year`), categorical variables with low cardinality (`sector`, `building_type`, `heating`), categorical variables with high cardinality (`neighborhood`, `street`), natural-language variables with free-text tags, and spatial.

The evaluation metric is Root Mean Squared Error (in EUR): $\mathrm{RMSE}(\mathbf{y}, \hat{\mathbf{y}}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$.

Rent prices are heavily right-skewed (Section 2.3), RMSE on the raw scale is dominated by a handful of luxury listings; this motivates training on the log-transformed target and inverse-transforming predictions (Section 4).

2 Exploratory Data Analysis

The EDA process is centered on questions that help make decisions about modeling. These questions include determining where the missing values lie, the nature of the target variable, the importance of numeric variables, the stratification of categorical variables by price, the nature of the spatial structure imposed by the city, and finally, the similarity of the train-test split.

2.1 Currency normalization

The original `price` feature in the train set is expressed in both EUR and RON currencies according to a `price_currency` indicator, while there is no currency indicator for the test dataset. When training without considering this, it ends up mixing two different currencies (which means the FX rate of RON/EUR ≈ 5.20), resulting in collapsing loss function. To address this, normalization is applied to all prices in EUR by considering prices with positive price indicators (and values larger than 2500) to be RON and dividing them by FX rates. Prices that are either negative or missing are excluded from analysis. After normalization, the median rent price tends to converge to ≈ 600 EUR, and the maximum gets reduced significantly.

2.2 Missing values

Figure 1 shows the missing-rate ranking of the 35 most-incomplete columns. Thirteen columns are essentially empty ($\geq 99\%$ missing): `extra_spaces`, `project_amenities`, `info_state_detail`, `rooms_number_range`, `area_range`, `number_of_floors_project`, `security`, `number_of_properties`, `offered_estates_type`, `state`, `floors_num`, `area_to`, `area_from`. These are dropped unconditionally before modeling. A second tier (`deposit`, `deposit_currency`, `remote_services`, `info_rent_to_students`) sits at 80–90% missing; they are kept because the non-missing slice is informative and tree-based models tolerate NaNs through median imputation. Core attributes such as `rooms`, `sector`, `floor`, and `neighborhood` have $< 10\%$ missing and are fully usable.

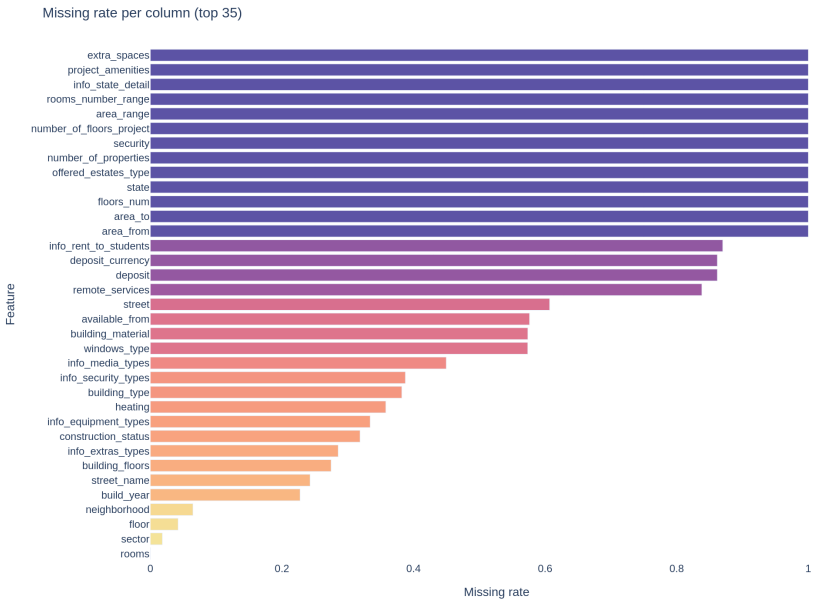


Figure 1: Missing rate per column, top 35. Bars at 1.0 are dropped before modeling; bars in the 0.2–0.6 range are kept and imputed.

2.3 Target distribution and skew

Figure 2 compares the raw EUR price distribution with its `log1p` transformation.

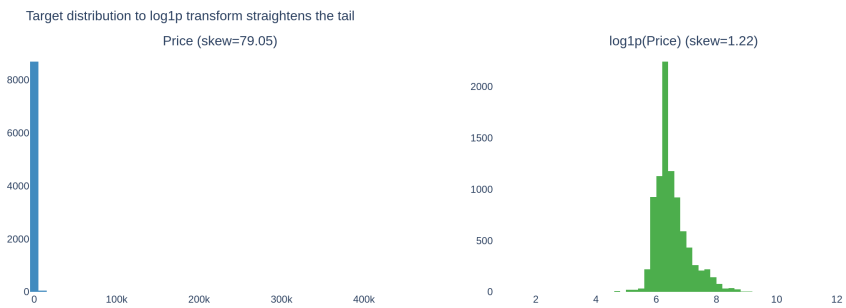


Figure 2: Target distribution. Left: raw `price` on EUR scale, with a pathological right tail. Right: `log1p(price)` is substantially more symmetric and is used as the training target.

The raw price distribution is extremely right-skewed, with skewness ≈ 79. This behavior is caused by a long tail of luxury listings and possible data-entry outliers, even after currency normalization. On this original scale, RMSE is dominated by a small number of large absolute errors in the tail and is therefore less sensitive to performance on the main rental market.

After applying `log1p`, the skewness decreases to approximately 1.2, and the bulk of the distribution becomes much closer to a Gaussian-like shape. Modeling on the log scale reframes squared errors in a roughly multiplicative way: for example, a 10% pricing error on a 600 EUR studio and a 10% error on a 3000 EUR penthouse become more comparable. This is more appropriate for rent prediction, where relative pricing errors are often more meaningful than raw absolute deviations.

The 1st, 5th, 50th, 95th, and 99th percentiles of EUR price are $\approx \{260, 320, 600, 1500, 2500\}$, indicating that most listings lie between roughly 300 EUR and 2500 EUR. Therefore, the training target is clipped at the 0.5th and 99.5th percentiles. This reduces the influence of residual data-entry outliers while preserving genuinely expensive listings.

2.4 Numeric distributions and outliers

Figure 3 depicts violin plots with the included boxplot plots for the 8 numeric features which pass through the missing-rate filter on the 0.5% to 99.5% central quantile scale. Issues have emerged with some of these features; `floor` features contain negative sentinel values (due to encoding), `building_floors` contains zero values, and `build_year` is not restricted to the interval $[1850, 2030]$. These issues are addressed through feature engineering where invalid.

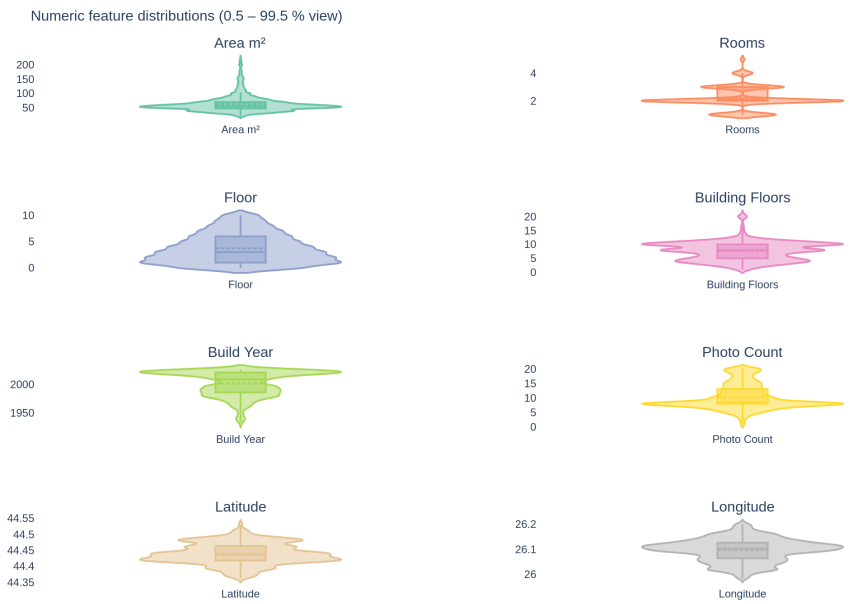


Figure 3: Distributions of numeric features (central 99% view). `area_m2` is right-skewed but bounded; `rooms` is discrete and well-behaved; `floor`, `building_floors`, `build_year` all need sentinel cleanup.

2.5 Numeric correlations

The correlation matrix for numerical variables in Figure 4 reveals that, at the level of raw values, there is not a single numerical variable with a high correlation to `price`, with the highest correlation being `rooms` with 0.10, followed by `latitude` with 0.07, and `area_m2` with 0.05. This is to be expected, as this is the primary reason why linear models perform poorly: the majority of the information is non-linear (like price = size × location × amenities) and resides in the categorical variables and their interactions. Among numeric variables, the most relevant internal correlations are:

- `rooms` ↔ `area_m2` (0.30): listings with more rooms generally have larger surface areas.
- `rooms` ↔ `photo_count` (0.32): larger or more complex listings tend to include more photos.
- `latitude` ↔ `longitude` (−0.25): Bucharest's elongated north-east to south-west spatial layout.

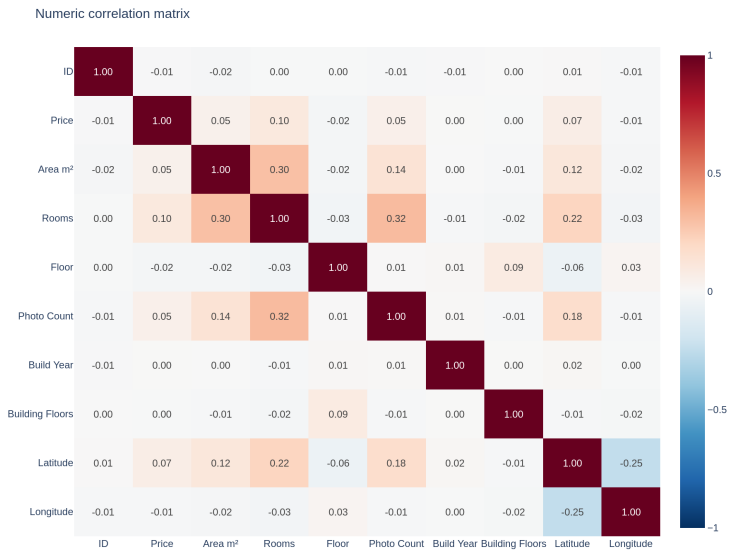
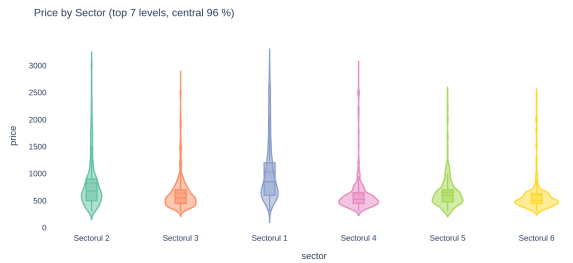


Figure 4: Pearson correlation between numeric features and the target. Linear correlations with `price` are weak ($|\rho| < 0.11$); most of the signal is non-linear and routed through categorical features.

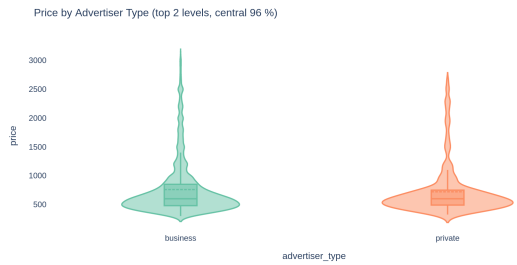
2.6 Categorical effects on price

Figure 5 presents violin plots showing how price differs across the 5 most insightful categorical variables (`sector`, `advertiser_type`, `building_type`, `heating`, `building_material`) by only considering the central 96% of prices to make it possible for the y-axis to fit.

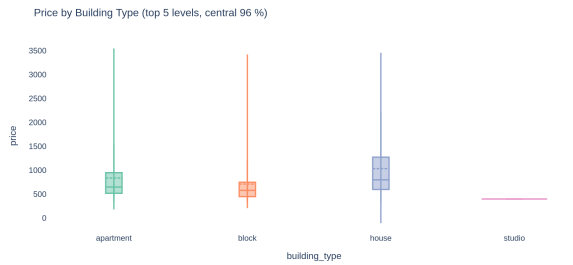
Sectors 1-3 tend to have larger medians compared to sectors 4-6; agency apartments are marginally more expensive; modern buildings, exclusive rich areas and district heating lead to greater central tendency. Violin plot shapes reveal additional information: the bimodal shape within `building_type` reveals two distinct rental regimes within the same category, which can be effectively used.



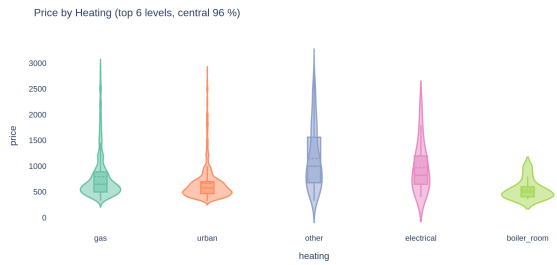
(a) Price by sector.



(b) Price by advertiser type.



(c) Price by building type.

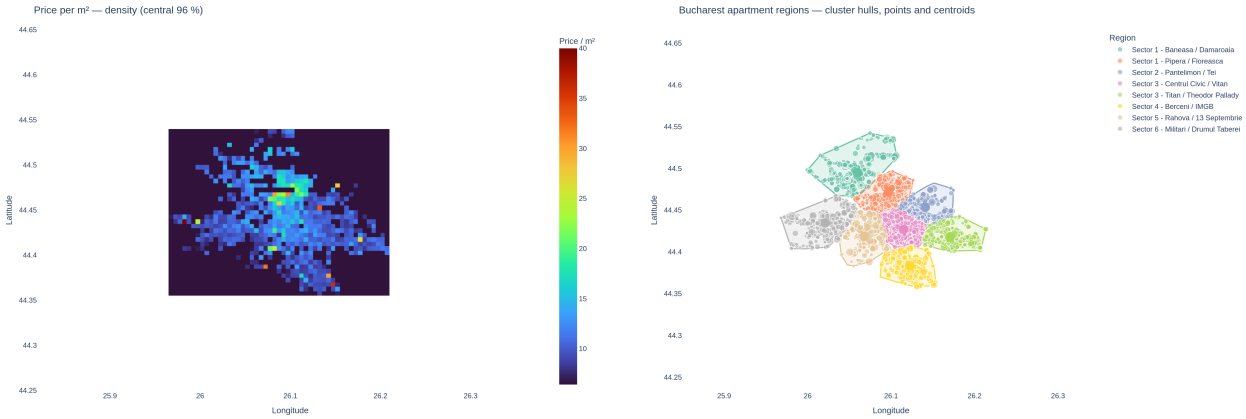


(d) Price by heating type.

Figure 5: Categorical features stratify price clearly. Violins show the within-level distribution; multimodality (e.g. in `building_type`) is the kind of signal trees capture but linear models miss.

2.7 Spatial structure

The price per square meter density for Bucharest is shown in Figure 6a, using the 2D histogram on `(longitude, latitude)`, with the mean `price_per_sqm` of the cells being its values. One can observe the natural trend of price distribution, where dense clusters appear in the city center (sectors 1 to 3, or the 44.45N-26.10E region), while there is sparse clusters in the low-priced outskirts. However, the raw coordinates are never fed into the model because they tend to introduce noise and overfit the exact locations. Instead, by applying K-Means to generate 8 regions based on the coordinates (Figure 6b), named according to their corresponding sectors and neighborhoods.
Tip: Consult the notebook for the full-resolution correlation and clustered heatmaps.



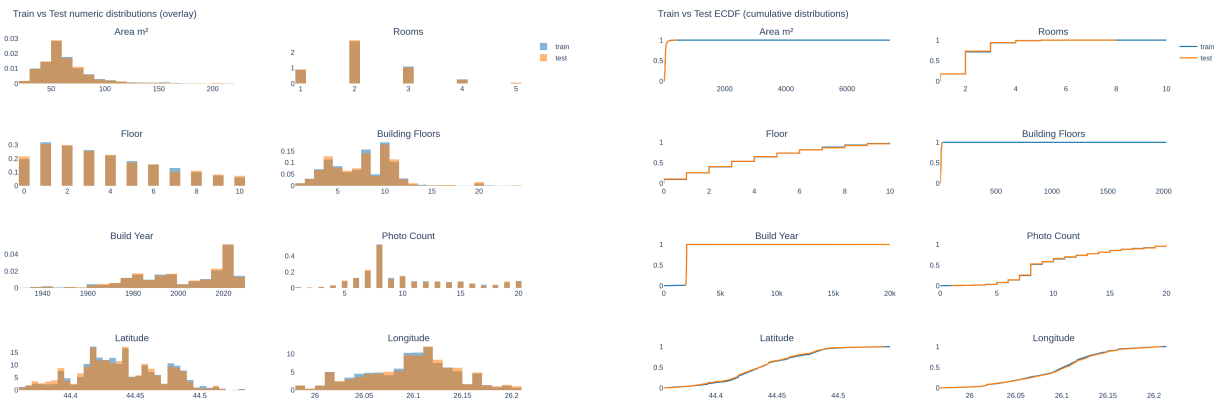
(a) Average price/m^2 density on a longitude/latitude grid. Hotspots align with the central sectors.

(b) K-Means region clusters. Each color is a region; large filled markers are cluster centroids.

Figure 6: Spatial structure of the rental market. The cluster label and centroid coordinates replace the raw `(latitude, longitude)` pair as model inputs.

2.8 Train vs Test drift

Modeling is preceded by an overlay of numeric distributions of train and test data (Figure 7a), and confirmed with an empirical CDF plot (Figure 7b). The overlay looks almost perfect: all medians change no more than single-digit percents in relation to each numeric feature, IQRs are similar, and ECDF plots look the same. This is encouraging news as it allows me to use the K-fold cross-validation estimate from my training data. If there was significant drift, adversarial validation or importance weighting techniques would be used instead.



(a) Histogram overlay (probability density).

(b) Empirical CDF, train vs test.

Figure 7: Train vs test covariate-shift diagnostics. Overlap is near-perfect across all numeric features, validating plain K-fold CV.

3 Data Preprocessing and Feature Engineering

3.1 Cleaning and target preparation

The cleaning pipeline consists of the following steps:

- Cast all numeric strings to floating-point values, including values that use European commas.
- Coerce boolean variables, like `is_private_owner` and `is_promoted`, to binary indicators in $\{0, 1\}$.
- Replace sentinel or invalid values with NaN, including `floor` < 0, `building_floors` ≤ 0, and `build_year` outside the interval $[1850, 2030]$.
- Drop the fully empty columns identified in Section 2, together with target-leakage columns such as `price_currency`, `deposit_currency`, and `available_from`.
- Drop free-text columns whose information is instead represented through amenity flags: `tags`, `info_security_types`, `info_extras_types`, `info_equipment_types`, and `info_media_types`.
- Clip the target variable at the 0.5th and 99.5th percentiles to reduce the influence of extreme residual outliers.

3.2 Feature engineering

I also engineer several domain-specific real-estate features:

- *Density:* `area_per_room` = `area_m2`/`rooms` and `rooms_density` = `rooms`/`area_m2`.
- *Floor position:* `floor_ratio` = `floor`/`building_floors`, plus `is_ground_floor` and `is_top_floor`.
- *Building age:* `building_age` = 2026 − `build_year`.
- *Spatial context:* K-Means cluster label `geo_territory_model_feature`, centroid coordinates, central-sector flag for Sectors 1–3, and a premium-neighborhood flag derived from names such as `aviatiei`, `herastrau`, `floreasca`, `primaverii`, `dorobanti`, `pipera`, `victoriei`, `cotroceni`, `unirii`, `tineretului`, and `kiseleff`.
- *Amenity flags:* regex indicators for furniture, parking, AC, balcony, elevator, security, storage, garden, washer, and dishwasher.

Raw `latitude`/`longitude` and `photo_count` are excluded because they risk micro-location overfitting and describe listing presentation rather than apartment value.

3.3 Preprocessing pipelines

Two versions of preprocessing are used depending on the model family:

- **Preprocessing for tree models.** It applies to Decision Tree, Random Forest, and XGBoost regressions. Numerical features are filled with median imputation. Categorical features are filled by constant value `Unknown`. Low cardinality features, which can take no more than 25 different values, are encoded via `OneHotEncoder` with `min_frequency=5` and `handle_unknown="ignore"` parameters. Features `sector` and `heating` are examples of such features. On the other hand, high cardinality features such as `neighborhood`, `street`, and `geo_territory_model_feature` are frequency encoded.
- **Preprocessing for linear models.** It applies to Ridge regression. The process of imputation, categorical features splitting, one-hot and frequency encodings is similar to the preprocessing for tree models but differs by numerical and frequency-encoded features standardization via `StandardScaler` due to the need in it for the linear.

The encoding works by assigning each level its frequency in the training set to its [0,1] interval or 0 if it is out-of-bounds, thus achieving higher compression ratios than one-hot (by about 1500 dimensions in the case of the combination `neighborhood`+`street`, since it encodes all the information into a single dimension). This section studies the performance gains of such choice.

4 Models

I evaluate four model families covering the spectrum from low to high capacity, the experimental comparison includes four regression models:

1. **Ridge regression:** the regularized linear baseline used to measure simple additive effects. It uses the linear preprocessor and ℓ_2 regularization with $\alpha = 2.0$.
2. **Decision Tree:** the single-tree nonlinear baseline used to capture simple feature interactions. It uses the tree preprocessor, with `max_depth=12` and `min_samples_leaf=5`.
3. **Random Forest:** the bagging-based ensemble. It uses 300 trees with `max_depth=22` and `min_samples_leaf=2`, together with the tree preprocessor.
4. **XGBoost:** the boosting-based ensemble. It uses the tree preprocessor and the following tuned configuration: 700 estimators, `learning_rate=0.04`, `max_depth=6`, `min_child_weight=2`, `subsample=0.85`, `colsample_bytree=0.85`, and `reg_lambda=0.5`.

All 4 are trained on $\log(1 + y)$ via `TransformedTargetRegressor`. Predictions are inverse-transformed with `expm1` and clipped at zero. The CV protocol is shuffled 5-fold KFold for every fold I record both the train-fold RMSE (model's error on its own training data) and the validation-fold RMSE (its honest generalization estimate). The gap between the two is the *generalization gap*.

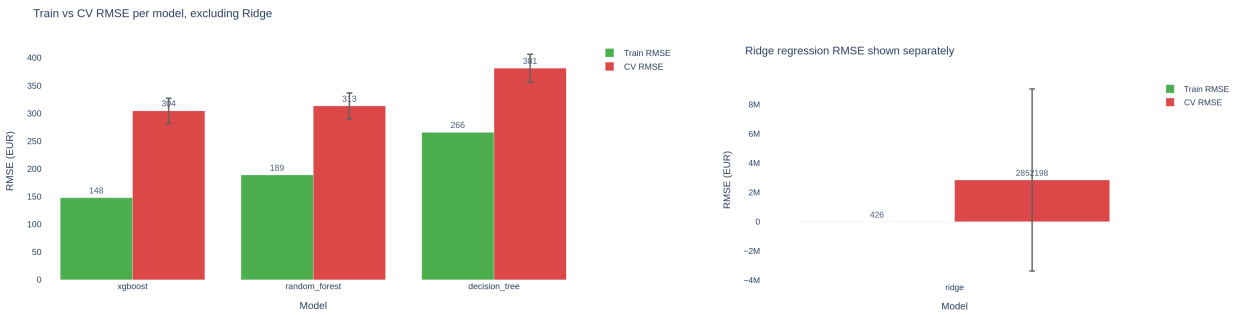
4.1 Cross-validated comparison

Table 1 summarizes the 5-fold CV results. Ridge is kept only as a baseline because its CV error is unstable and much larger than that of the tree-based models.

Model	Train RMSE	CV RMSE	CV std	Gap	Gap %
XGBoost	147.9	304.5	22.8	156.6	105.9
Random Forest	188.9	313.3	23.2	124.4	65.8
Decision Tree	265.7	381.3	25.2	115.6	43.5
Ridge regression	425.8	2.85×10^6	–	–	–

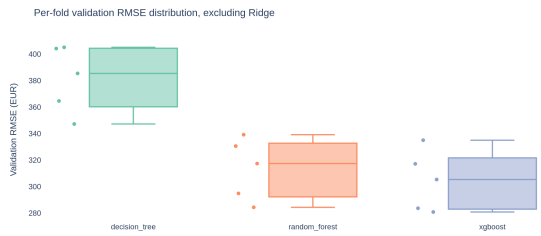
Table 1: 5-fold CV results in EUR. Gap is defined as CV RMSE − Train RMSE, and Gap% as Gap/Train RMSE × 100. Ridge regression is separated because its divergent CV RMSE would distort the comparison scale and indicates that the linear baseline is not reliable for this feature space.

Reading the table. XGBoost wins decisively on CV RMSE, with the largest generalization gap as expected: a flexible boosted tree memorizes the training set easily, but its leaf-level regularization keeps the validation error low. Random Forest is ~ 9 EUR behind; the single decision tree is ~ 80 EUR worse. Ridge collapses: a 99% variance ratio across folds shows that one fold contains a leverage point that one-hot encoding stretches into a single dimension the model cannot regularize away. Ridge is therefore plotted separately (Figure 8b) and excluded from headline comparisons.

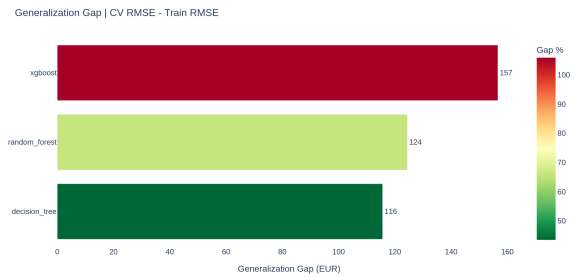


(a) Train vs CV RMSE per model. **(b)** Ridge, CV RMSE is on a different scale.

Figure 8: Per-model train versus 5-fold CV RMSE. Ridge's CV error is $\sim 10^4\times$ the others; the rest of the report works with the three tree-based models.



(a) Per-fold validation RMSE distribution.

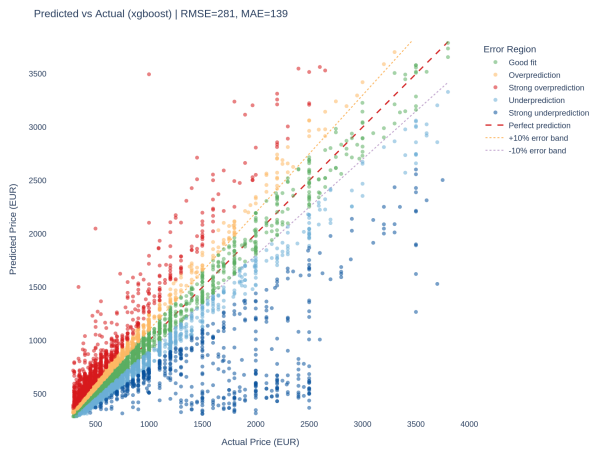


(b) Generalization gap (CV − Train).

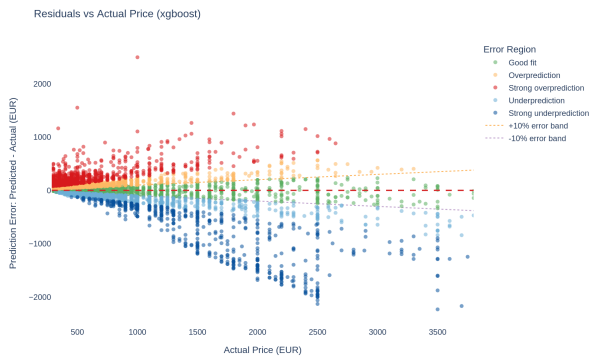
Figure 9: (a) The fold-level RMSE spread is comparable across models ($\sigma \approx 20$–25 EUR), so no model is pathologically fold-sensitive. (b) The gap-vs-Gap% view shows boosting both *wins on CV* and *overfits the most in absolute EUR*; the latter is corrected in tuning (§5).

4.2 Diagnostic plots on the best single model

The strongest model, XGBoost, is further analyzed in Figure 10. The two diagnostic plots compare predicted rents against actual rents and show how the residual errors vary across the full price range, making it possible to assess both calibration in the common rental segment and failure patterns in the expensive tail.



(a) Predicted vs actual price.



(b) Residuals vs actual price.

Figure 10: XGBoost diagnostics. Predicted-vs-actual is near-diagonal in the bulk; the residual cloud is broadly symmetric with mild under-prediction at the high tail.

The predicted-vs-actual scatter confirms that the model is well calibrated for the main rental interval, roughly 400–2500 EUR, where most listings are located. In this region, points remain close to the diagonal, meaning that predicted prices are close to the observed rents. The spread increases for more expensive apartments, especially above approximately 2500 EUR, where the training data becomes sparse and the model has fewer comparable examples.

The residual plot supports the same interpretation. Residuals are approximately centered around zero for typical listings, which indicates that the model does not show a strong systematic bias in the bulk of the market.

For high-rent listings, residuals become slightly negative, showing a tendency to under-predict luxury apartments. This is expected for a log-target model trained on a distribution with a thin right tail: the model favors stable predictions for common listings and is more conservative on rare, very expensive examples.

5 Hyperparameter Tuning

I tune the two strongest models with `RandomizedSearchCV`, using the same 5-fold KFold splitter as before and optimizing `neg_root_mean_squared_error`. Because the search spaces are relatively large, the tuning setup is split into two tables: Table 2 reports the searched ranges, while Table 3 reports the selected configurations.

Model	Hyperparameter	Search values
Random Forest	`n_estimators`	$\{300, 500\}$
Random Forest	`max_depth`	$\{16, 22, \text{None}\}$
Random Forest	`min_samples_split`	$\{2, 5, 10\}$
Random Forest	`min_samples_leaf`	$\{1, 2, 4\}$
Random Forest	`max_features`	$\{\text{sqrt}, 0.6, 0.8\}$
XGBoost	`n_estimators`	$\{500, 800, 1200\}$
XGBoost	`learning_rate`	$\{0.02, 0.035, 0.05\}$
XGBoost	`max_depth`	$\{4, 6, 8\}$
XGBoost	`min_child_weight`	$\{1, 2, 4\}$
XGBoost	`subsample`	$\{0.75, 0.85, 1.0\}$
XGBoost	`colsample_bytree`	$\{0.75, 0.85, 1.0\}$
XGBoost	`reg_lambda`	$\{0.1, 0.5, 1.0\}$

Table 2: RandomizedSearchCV search spaces for the two strongest models.

Model	Best parameters	CV RMSE
Random Forest	`trees=300, depth=22, split=10, leaf=1, maxfeat=0.8`	316.30 EUR
XGBoost	`trees=1200, lr=0.02, depth=8, child=1, subs=0.75, col=1.0,` λ`=0.1`	300.90 EUR

Table 3: Best RandomizedSearchCV configurations using 5-fold CV and RMSE scoring.

The tuned XGBoost improves the default configuration by approximately 4 EUR RMSE, whereas Random Forest gains almost nothing, indicating that it was already close to its best configuration. The search budget is intentionally modest: with fold-to-fold variation around 20 EUR, a much larger grid would mostly chase differences smaller than the validation noise.

6 Ablation Study

To measure the effect of the main modeling choices, I run four controlled ablation variants. The learning algorithm and the 5-fold CV protocol are kept fixed, while one component of the pipeline is changed at a time. The goal is to determine whether the performance gains come mainly from the target transformation, the engineered features, or the categorical encoding strategy.

Variant	Description
`full_features + log_target`	Reference config using all engineered features and the `log1p` target.
`full_features + raw_target`	Same feature set as the reference setup, but trained directly on price EUR.
`base_features + log_target`	Removes engineered variables such as `floor_ratio`, `building_age`, `is_central_sector`, premium-neighborhood flags, and amenity.
`ohe_only_high_card + log`	Replaces frequency encoding for high-cardinality variables such as `neighborhood` and `street` with plain one-hot encoding.

Table 4: Definition of the controlled ablation variants. Each row modifies one pipeline component while keeping the XGBoost model and cross-validation protocol fixed, allowing the effect of target transformation, feature engineering, and categorical encoding to be evaluated separately.

Variant	Train RMSE	CV RMSE	CV std	Gap
`full_features + raw_target`	119.1	302.5	19.9	183.3
`full_features + log_target`	147.9	304.5	22.8	156.6
`base_features + log_target`	158.2	307.6	22.8	148.4
`ohe_only_high_card + log`	189.2	308.1	24.0	118.9

Table 5: Ablation results for the XGBoost pipeline. CV RMSE is the main comparison metric, while Train RMSE and Gap indicate bias–variance behavior.

From the ablation results, it can be seen that all four models have similar CV RMSE scores in the range of 302 to 308 EUR. However, the raw-target variant is the most accurate one, even though it has the biggest difference between training and validation, which means that the model tends to overfit data with extreme price values.

The log-target model has a slightly lower CV RMSE than the raw-target one, yet it is more robust and suitable for a skewed target distribution in the rental market. The removal of engineered features raises CV RMSE, which indicates that such features contain valuable information for prediction. Finally, frequency encoding proves to be preferable to one-hot encoding due to the higher complexity of the latter.

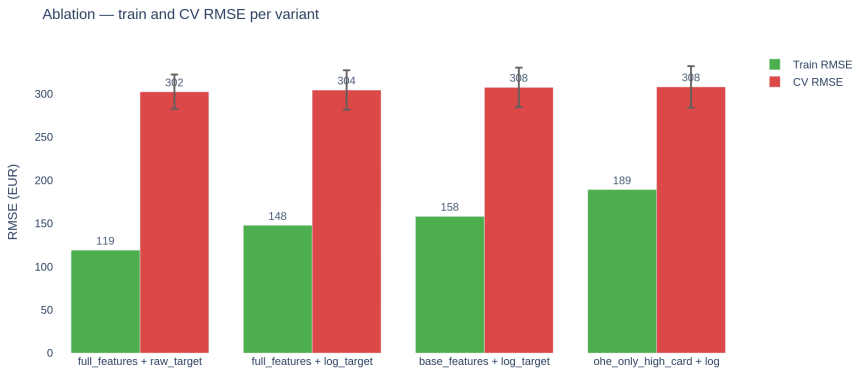


Figure 11: Train and CV RMSE across ablation variants. The validation scores remain close, while the training errors differ more substantially.

Reading the ablation. Two findings:

- *The log target mainly reduces overfitting.* It increases train RMSE from 119 to 148, but lowers the train–CV gap from 183 to 157. CV RMSE changes only slightly (302 → 305), so the log transform is not strictly best for RMSE, but gives a more stable model.
- *Feature engineering and frequency encoding help modestly.* Removing engineered features adds about 3 EUR CV RMSE, while replacing frequency encoding with full OHE adds about 4 EUR and increases dimensionality. Each effect is small compared with fold variance, but together they provide a useful gain.

7 Final Ensemble and Validation

This last predictor is built as a weighted combination of three types of XGBoost models. It is not our aim to build an ensemble using vastly different modeling techniques, but to construct one using boosting techniques of varying bias-variance trade-offs. Weights are obtained using out-of-fold minimum RMSE on the training data, as before.

- `tuned`: the best XGBoost configuration found during hyperparameter search in Section 5. This is the main model and provides the strongest individual validation performance.
- `fast`: a more regularized boosting model, with `n_estimators=400`, `learning_rate=0.06`, and `max_depth=4`. It is included as a low-complexity component that may improve robustness if the deeper models overfit.
- `deep`: a higher-capacity model, with `n_estimators=1100`, `learning_rate=0.025`, `max_depth=8`, and `min_child_weight=1`. It is included to capture more complex nonlinear interactions that the tuned model may underfit.

Ensemble weights are sought using a coarse grid with step size 0.1. The optimal weights are given by $0.7 \cdot \texttt{tuned} + 0.0 \cdot \texttt{fast} + 0.3 \cdot \texttt{deep}$, resulting in **ensemble OOF RMSE = 301.21 EUR**. This outcome reveals that the tuned variant is carrying the lion's share of the predictive power, with the deep model adding an auxiliary correction of a somewhat smaller magnitude. No weight is attributed to the fast version, implying that combining this model with the other two models already yields sufficient out-of-fold improvements.

Holdout sanity check. In order to ensure that the cross-validation estimate is realistic enough, the same blend is assessed using an independent 80/20 holdout split. The resulting holdout RMSE is **328.48 EUR** vs. OOF RMSE = 301.21 EUR. The gap of roughly 27 EUR between those values is reasonable, as a single 20% holdout split has a larger variance compared to the five-folds CV average. The gap is also very close to the OOF RMSE's observed per-fold standard deviation of around 22 EUR.

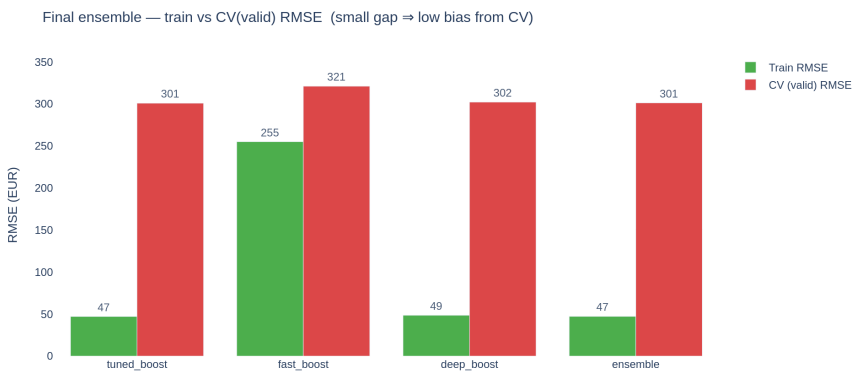


Figure 12: Train vs CV RMSE for the 3 XGBoost component models and the final ensemble. The higher-capacity boosting models reach very low train error, while the validation error remains around 301 EUR.

Three conclusions follow from the ensemble validation:

- **The optimized XGBoost model is still the primary one.** With a weight of 0.7, it proves that the search for hyperparameters managed to find an appropriate trade-off between model complexity and its generalizing ability.
- **The deep model provides a useful structure supplement.** Despite having higher complexity and lower training error, it is not applied independently; instead, its weight of 0.3 implies that it possesses some additional non-linear structures useful when averaging.
- **The fast model is not necessary in the final blend.** The best weight for it is equal to 0.0, which proves that the shallow model does not bring any improvements after two others are considered.

8 Feature Importance

The top 25 features in Figure 13 belong to the final XGBoost tuning step. Premium neighborhood stands out from all other features, by a large margin, being a simple binary flag ($\approx$ 28% of the total importance) for neighborhoods where houses have high prices in Bucharest. After this variable, comes the list of physical characteristics (`rooms`, `area_m2`, `deposit`). The rest of the variables from the top of the distribution are related to centrality sector flags (`Sector Sectorul 1`, `is_central_sector`), construction flags (`building_material`, `build_year`), and amen

Note: `is_premium_neighborhood` subsumes a chunk of the `neighborhood`-frequency-encoded signal: when both are present, the regex flag captures the strongest sub-population and the rest of `neighborhood` contributes the residual.

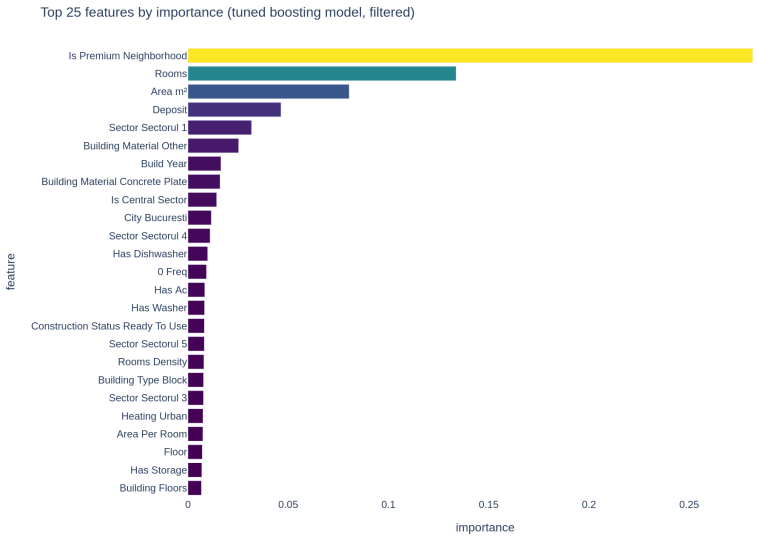


Figure 13: Top 25 XGBoost feature importances. The hand-crafted `is_premium_neighborhood` flag is the single strongest predictor; raw physical attributes (`rooms`, `area_m2`) follow.

Limitations and natural extensions. This current approach exhibits residual fan-out for large price points (cf. Figure 10b); an appropriately protected target-encoded `neighborhood` feature, instead of `is_premium_neighborhood`, would likely provide additional geographic signals currently captured by a simple dummy variable.

The second direction would be features based on images (`photo_count` is present in the dataset, but no pictures themselves), because listing appearance does play a role in Bucharest real estate.

Lastly, there might be opportunities in external macroeconomic factors (rates, inflation, seasonality), which, however, are not included in this dataset. The improvement of these approaches would not be able to bring the model out from $\sigma \approx 22$ EUR per fold noise floor for more than a few EUR, but on Kaggle they would definitely pay off.

9 Conclusions

Summary. With the Bucharest apartment rental data, the best results are obtained by gradient-boosted trees (XGBoost), achieving approximately 301 EUR in terms of 5-fold CV RMSE on a target that varies between approximately 750 EUR in the interquartile range.

The next best model is the random forest at approximately 5% loss compared to XGBoost. A single decision tree is 25% worse than XGBoost. Finally, an L2 linear regression model fails to solve the problem (its CV RMSE is undefined because of divergence due to one-hot leverage working against unregularized direction shrinkage).

What moved the needle.

- **Currency normalization**: without it, the model literally cannot fit, because the target is in two scales differing by 5.2×.
- **Engineered features**: worth ≈ 3 EUR CV RMSE (`is_premium_neighborhood`, the floor ratios, building age, and amenity flags collectively).
- **Frequency encoding for high-cardinality categoricals**: worth ≈ 4 EUR CV RMSE versus plain one-hot, plus a major dimensionality saving.
- **Spatial clustering**: replacing raw `(lat, lon)` with 8-cluster K-Means labels and centroid coordinates lets the boosting tree split cleanly on regions without overfitting micro-locations.
- **Log-target transform**: not strictly required for headline RMSE, but reduces the train-CV gap from 183 to 157 EUR and stabilizes residuals at the high-price tail.